Machine Learning Notes

Algorithms, workflow, evaluation, and responsible use

1. What is Machine Learning?

Machine Learning (ML) is a branch of AI that builds systems which learn patterns from examples. A model uses input features to produce predictions or discover structure.

- Supervised learning uses labelled examples.
- Unsupervised learning finds patterns without target labels.
- Reinforcement learning learns actions through rewards and feedback.

Model performance depends on data quality, task definition, evaluation, and real-world use. A model is not automatically correct or fair.

2. Supervised Learning Algorithms

Regression and classification

Regression predicts numerical outcomes such as price or energy consumption. Classification predicts categories such as spam/not spam or a disease class.

- Linear regression: predicts a numerical value using a linear relationship.
- Logistic regression: estimates class probabilities, commonly for classification.
- Decision tree: makes predictions using feature-based splits.
- Random forest: combines multiple decision trees.
- K-nearest neighbours: predicts using nearby examples.
- Support Vector Machine: finds a separating boundary with a margin.
- Naive Bayes: applies Bayes' theorem with simplifying assumptions.

3. Unsupervised Learning

- K-means: groups observations around k centroids; scaling often matters.
- Hierarchical clustering: creates a nested tree of clusters.
- DBSCAN: finds dense groups and can identify noise.
- PCA: reduces dimensionality by finding directions of high variance.

Results can change with distance measures, feature scales, and parameter settings. Clusters need interpretation in the context of the problem.

4. ML Workflow and Preprocessing

1. Define the target and evaluation metric.
2. Collect representative data and inspect quality.
3. Split data into training, validation, and test sets as appropriate.
4. Build a simple baseline.
5. Handle missing values and encode categorical variables.
6. Train candidate models using training data.
7. Tune using validation data or cross-validation.
8. Evaluate on held-out test data.

9 Document limitations and monitor performance after deployment.

- Feature scaling can help distance-based algorithms.
- One-hot encoding is a common way to represent categories.
- Feature engineering creates useful variables from existing information.
- Data leakage occurs when training uses information unavailable at prediction time.
- Fit transformations on training data and apply them consistently to other data.

5. Evaluation Metrics

Classification

- Accuracy = correct predictions / all predictions.
- Precision = TP / (TP + FP): among predicted positives, how many are correct?
- Recall = TP / (TP + FN): among actual positives, how many are found?
- F1-score is the harmonic mean of precision and recall.
- A confusion matrix counts true positives, true negatives, false positives, and false negatives.

Regression

- MAE: mean absolute prediction error.
- MSE: mean squared error; penalizes large errors more strongly.
- RMSE: square root of MSE and uses the target's units.
- R-squared compares model fit with a mean-based baseline; interpret in context.

Accuracy alone can be misleading for rare events or imbalanced classes. Select metrics based on the cost of different errors.

6. Overfitting, Underfitting, and Responsible ML

Overfitting occurs when a model learns training-specific noise and performs poorly on new data. Underfitting occurs when a model is too simple to capture the pattern.

- Use validation and cross-validation appropriately.
- Control model complexity or use regularization.
- Compare training and validation performance.
- Check for data drift after deployment.
- Protect confidential data and examine unequal error patterns.
- Document assumptions, limitations, and uncertainty.

7. Simple Python Example

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score

data = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
data.data, data.target, test_size=0.2,
random_state=42, stratify=data.target
)
model = RandomForestClassifier(random_state=42)
model.fit(X_train, y_train)
pred = model.predict(X_test)
print("Accuracy:", accuracy_score(y_test, pred))
```

Install scikit-learn if needed with: pip install scikit-learn. A single test score is only a demonstration, not a guarantee of future performance.